REST API 엔드포인트 명세서

개요

Irys 체인 API 서버는 RESTful HTTP API를 통해 블록체인 데이터에 대한 접근을 제공합니다. 모든 API 엔드포인트는 `/v1` 경로 하위에 위치하며, JSON 형식으로 데이터를 주고받습니다.

기본 URL: `http://{host}:{port}/v1`

인증 및 보안

서명 기반 인증

- 트랜잭션 제출 시 Irys 서명 스키마를 사용한 디지털 서명 필요
- 서명은 트랜잭션 헤더의 해시값에 대해 생성
- 공개키를 통한 서명 검증으로 트랜잭션 무결성 보장

CORS 설정

- 모든 오리진에서의 요청 허용 (개발 환경)
- 프로덕션 환경에서는 적절한 CORS 정책 적용 권장

공통 응답 형식

성공 응답

```
{
```

```
"data": { /* 응답 데이터 */ },
"status": "success"
}
```

오류 응답

```
{
"ErrNoId": {
"id": "요청한 리소스 ID",
"err": "오류 설명"
}
}
```

또는

```
{
"Internal": {
"err": "내부 서버 오류 설명"
}
}
```

API 엔드포인트

1. 노드 정보

GET /info

노드의 현재 상태 및 기본 정보를 반환합니다.

응답 예시:

```
{
"version": "0.0.1",
"peerCount": 5,
"chainId": 1270,
"height": 12345,
"blockHash": "0x1234567890abcdef...",
"blockIndexHeight": 12340,
"blocks": 12345,
"isSyncing": false,
"currentSyncHeight": 0
}
```

응답 필드:

- `version` : 노드 소프트웨어 버전
- `peerCount` : 연결된 피어 수
- `chainId` : 체인 식별자
- `height` : 현재 블록 높이
- `blockHash` : 최신 블록 해시
- `blockIndexHeight` : 블록 인덱스 높이
- `blocks` : 총 블록 수

- `isSyncing` : 동기화 진행 여부
- `currentSyncHeight` : 현재 동기화 목표 높이

2. 트랜잭션 관리

POST /tx

새로운 데이터 트랜잭션을 메모리풀에 제출합니다.

요청 본문:

```
{
"id": "0x1234567890abcdef...",
"signature": "0xabcdef1234567890...",
"owner": "0x742d35Cc6634C0532925a3b8D4C0d886E4C2e7b9",
"target": null,
"anchor": "0x0000000000000000000000000000000000000000000000000000000000000000",
"data_size": 1024,
"data_root": "0xfedcba0987654321...",
"ledger_id": 1,
"tags": []
}
```

응답:

- `200 OK` : 트랜잭션이 성공적으로 처리됨
- `400 Bad Request` : 잘못된 서명, 잘못된 앵커, 또는 검증 오류
- `402 Payment Required` : 자금 부족

- `500 Internal Server Error` : 내부 서버 오류

오류 코드:

- `InvalidSignature` : 서명이 유효하지 않음
- `Unfunded` : 계정에 충분한 자금이 없음
- `InvalidAnchor` : 앵커 값이 유효하지 않음
- `CommitmentValidationError` : 커밋먼트 검증 실패

GET /tx/{tx_id}

트랜잭션 ID로 트랜잭션 정보를 조회합니다.

경로 매개변수:

- `tx_id` : 트랜잭션 해시 (H256 형식)

응답 예시:

```
{
"Storage": {
"id": "0x1234567890abcdef...",
"signature": "0xabcdef1234567890...",
"owner": "0x742d35Cc6634C0532925a3b8D4C0d886E4C2e7b9",
"data_size": 1024,
"data_root": "0xfedcba0987654321...",
"ledger_id": 1
}
}
```

GET /tx/{tx_id}/is_promoted

트랜잭션이 프로모션되었는지 확인합니다.

응답:

```
true
```

GET /tx/{tx_id}/local/data_start_offset

트랜잭션의 로컬 데이터 시작 오프셋을 반환합니다.

응답 예시:

```
{
"dataStartOffset": "1024"
}
```

3. 청크 관리

POST /chunk

데이터 청크를 메모리풀에 제출합니다.

요청 본문:

```
{
"data_root": "0xfedcba0987654321...",
"data_size": 1024,
"data_path": "base64_encoded_proof",
"bytes": "base64_encoded_data",
```

```
"tx_offset": 0
}
```

응답:

- `200 OK` : 청크가 성공적으로 처리됨
- `400 Bad Request` : 잘못된 증명, 데이터 해시, 청크 크기 등
- `500 Internal Server Error` : 내부 서버 오류

오류 코드:

- `InvalidProof` : 증명이 유효하지 않음
- `InvalidDataHash` : 데이터 해시가 일치하지 않음
- `InvalidChunkSize` : 청크 크기가 유효하지 않음
- `UnknownTransaction` : 알 수 없는 트랜잭션

GET /chunk/ledger/{ledger_id}/{ledger_offset}

원장 오프셋으로 청크를 조회합니다.

경로 매개변수:

- `ledger_id` : 원장 식별자 (u32)
- `ledger_offset` : 원장 내 오프셋 (u64)

응답 예시:

```
{
"Packed": {
"data": "base64_encoded_chunk_data",
"proof": "base64_encoded_proof"
```

```
}
}
```

GET /chunk/data_root/{ledger_id}/{data_root}/{offset}

데이터 루트와 오프셋으로 청크를 조회합니다.

경로 매개변수:

- `ledger_id` : 원장 식별자 (u32)
- `data_root` : 데이터 루트 해시 (H256)
- `offset` : 청크 오프셋 (u32)

4. 블록 정보

GET /block/{block_tag}

블록 태그로 블록 정보를 조회합니다.

경로 매개변수:

- `block_tag` : 블록 식별자
- `latest` : 최신 블록
- `{height}` : 블록 높이 (숫자)
- `{hash}` : 블록 해시 (base58 인코딩)

응답 예시:

```
{
"irys": {
"block_hash": "0x1234567890abcdef...",
```

```
"height": 12345,
"timestamp": 1640995200,
"previous_block": "0xfedcba0987654321...",
"evm_block_hash": "0xabcdef1234567890..."
},
"execution": {
"header": {
"number": 12345,
"hash": "0xabcdef1234567890...",
"parent_hash": "0xfedcba0987654321..."
},
"transactions": [
"0x1111111111111111...",
"0x2222222222222222..."
]
}
}
```


GET /block_index

블록 인덱스 정보를 조회합니다.

쿼리 매개변수:

- `height` : 시작 높이 (선택사항)
- `limit` : 반환할 항목 수 (선택사항)

응답 예시:

```
[
{
"height": 12345,
"block_hash": "0x1234567890abcdef...",
"timestamp": 1640995200
}
]
```

5. 가격 정보

GET /price/{ledger}/{size}

데이터 저장 가격을 조회합니다.

경로 매개변수:

- `ledger` : 원장 타입 (1: Publish, 2: Submit)
- `size` : 저장할 데이터 크기 (바이트)

응답 예시:

```
{
"permFee": "1000000000000000000",
"termFee": "1000000000000000",
"ledger": 1,
"bytes": 1024
```

```
}
```

GET /price/commitment/stake

스테이킹 가격 정보를 조회합니다.

응답 예시:

```
{
"value": "1000000000000000000",
"fee": "1000000000000000",
"userAddress": null
}
```

GET /price/commitment/pledge/{user_address}

사용자별 플레지 가격을 조회합니다.

경로 매개변수:

- `user_address` : 사용자 주소 (Ethereum 주소 형식)

응답 예시:

```
{
"value": "1000000000000000000",
"fee": "1000000000000000",
"userAddress": "0x742d35Cc6634C0532925a3b8D4C0d886E4C2e7b9"
}
```

6. 네트워크 정보

GET /peer_list

연결된 피어 목록을 반환합니다.

응답 예시:

```
[
{
"gossip": "192.168.1.100:8080",
"api": "192.168.1.100:8081",
"execution": {
"address": "192.168.1.100:30303",
"node_id": "0x1234567890abcdef..."
}
}
]
```

GET /network/config

네트워크 설정 정보를 반환합니다.

응답 예시:

```
{
"chain_id": 1270,
"consensus": {
```

```
"chunk_size": 262144,
"vdf_difficulty": 1000
}
}
```

7. 커밋먼트 트랜잭션

POST /commitment_tx

커밋먼트 트랜잭션을 제출합니다.

요청 본문:

```
{
"id": "0x1234567890abcdef...",
"signature": "0xabcdef1234567890...",
"owner": "0x742d35Cc6634C0532925a3b8D4C0d886E4C2e7b9",
"commitment_type": "Stake",
"value": "1000000000000000000"
}
```

8. 버전 정보

POST /version

피어 버전 정보를 교환합니다.

요청 본문:

```
{
"version": "0.1.0",
"protocol_version": 1,
"mining_address": "0x742d35Cc6634C0532925a3b8D4C0d886E4C2e7b9",
"chain_id": 1270,
"address": {
"gossip": "192.168.1.100:8080",
"api": "192.168.1.100:8081",
"execution": {
"address": "192.168.1.100:30303",
"node_id": "0x1234567890abcdef..."
}
},
"timestamp": 1640995200000,
"user_agent": "irys-node/0.1.0 (linux/x86_64)",
"signature": "0xabcdef1234567890..."
}
```


9. 앵커 정보

GET /anchor

현재 앵커 정보를 반환합니다.

응답 예시:

```
{
"anchor": "0x1234567890abcdef...",
"height": 12345,
"timestamp": 1640995200
}
```

10. 프록시 엔드포인트

ALL /execution-rpc

Ethereum RPC 호출을 Reth 노드로 프록시합니다.

설명:

- 모든 HTTP 메서드 지원 (GET, POST, PUT, DELETE 등)
- 요청 헤더와 본문을 그대로 전달
- Reth 노드의 응답을 클라이언트에게 스트리밍

오류 처리

HTTP 상태 코드

- `200 OK` : 요청 성공
- `400 Bad Request` : 잘못된 요청 (유효하지 않은 데이터, 서명 오류 등)
- `402 Payment Required` : 자금 부족
- `404 Not Found` : 요청한 리소스를 찾을 수 없음
- `500 Internal Server Error` : 내부 서버 오류

일반적인 오류 시나리오

1. **트랜잭션 제출 실패**

- 서명 검증 실패
- 자금 부족
- 잘못된 앵커 값
- 데이터베이스 오류

2. **청크 제출 실패**

- 잘못된 증명
- 데이터 해시 불일치
- 알 수 없는 트랜잭션
- 청크 크기 오류

3. **조회 실패**

- 존재하지 않는 리소스
- 데이터베이스 연결 오류
- 권한 부족

사용 예시

트랜잭션 제출 워크플로우

1. **가격 조회**

```
curl -X GET "http://localhost:8081/v1/price/1/1024"
```

2. **트랜잭션 생성 및 서명** (클라이언트 측)

3. **트랜잭션 제출**

```
curl -X POST "http://localhost:8081/v1/tx" \
-H "Content-Type: application/json" \
-d '{"id":"0x...","signature":"0x...","owner":"0x...","data_size":1024,...}'
```

4. **청크 제출**

```
curl -X POST "http://localhost:8081/v1/chunk" \
-H "Content-Type: application/json" \
-d '{"data_root":"0x...","data_size":1024,"bytes":"base64data",...}'
```

5. **트랜잭션 상태 확인**

```
curl -X GET "http://localhost:8081/v1/tx/0x1234567890abcdef..."
```

성능 고려사항

요청 제한

- JSON 페이로드 최대 크기: 1MB
- 요청 타임아웃: 5초 (클라이언트 기본값)
- 동시 연결 수: 서버 설정에 따라 제한

최적화 권장사항

- 대용량 데이터는 청크 단위로 분할하여 전송

- 배치 요청 시 적절한 간격으로 요청 분산
- 연결 재사용을 위한 Keep-Alive 헤더 사용
- 압축 지원 활용 (gzip, deflate)

보안 고려사항

입력 검증

- 모든 입력 데이터에 대한 엄격한 검증
- SQL 인젝션 및 XSS 공격 방지
- 적절한 데이터 크기 제한

인증 및 권한

- 트랜잭션 서명 검증 필수
- 피어 인증을 통한 네트워크 보안
- 적절한 CORS 정책 적용

모니터링

- API 호출 로깅 및 모니터링
- 비정상적인 트래픽 패턴 감지
- 오류율 및 응답 시간 추적